2025

Exploring Brent-Kung Adders in Balanced Ternary CMOS Logic

Astrea J.C. Rojas

Columbus State University

EXPLORING BRENT-KUNG ADDERS IN BALANCED TERNARY CMOS LOGIC

A THESIS SUBMITTED TO

THE D. ABBOTT TURNER COLLEGE OF BUSINESS AND TECHNOLOGY

IN PARTIAL FULFILLMENT OF

THE REQUIREMENTS FOR THE DEGREE OF

MASTER OF APPLIED COMPUTER SCIENCE

TSYS SCHOOL OF COMPUTER SCIENCE

BY

Astrea J. C. Rojas

Columbus, Georgia

November 2025

EXPLORING BRENT-KUNG ADDERS IN BALANCED TERNARY CMOS LOGIC

By

Astrea J. C. Rojas

Approved by

Committee Chair:

Dr. Hyrum D. Carroll

Committee Members:

Dr. Lixin Wang

Dr. Rodrigo Obando

Columbus State University

November 2025

ABSTRACT

The modern computer operates on a 64-bit architecture. These devices can store large numbers and precise decimals, but more advanced devices are needed to support progressing technologies every day. A more efficient system with higher speeds and larger operable numbers would be a key to optimization of computation as we know it. The ternary device, operating in base-3, has the potential to be that optimization. However, binary technology has such precedent and research that it is a difficult gap to span to compare the ternary system to the modern binary system. With a more advanced adder and optimized gates using the CMOS implementation of ternary logic, that gap becomes smaller.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS AND ABBREVIATIONS

CNTFET	Carbon Nanotube Field Effect Transistors
CMOS	Complementary Metal-Oxide Semiconductor
PTL	Pass Transistor Logic
BKA	Brent-Kung Adder

1. INTRODUCTION

1.1. Problem definition

Our progress in technology is limited by the speed that our computer can run, and the complexity of operations that they support. For decades the computer has operated on a binary system of on and off. These allow us to compute operations in binary, a base-2 system of operations. This system has functioned well so far, but there are possibilities for improvement. Here I seek to provide evidence that a potential avenue of that improvement lies in the realm of base-3.

The realization of this base-3 system of computing has multiple different methods of implementation, chief among them are Optical Ternary Computers[1], carbon nanotube field effect transistors (CNTFET), and complementary metal-oxide semiconductor (CMOS) implemented.[2][3] The Optical Ternary Computer uses light, liquid crystals, and polarized lenses to store and complete operations for a balanced ternary system. The CNTFET uses carbon nanotubes to store multiple levels of charges for an imbalanced ternary system.

Among the methods mentioned, the most widely manufacturable one is CMOS.[2] Due to this reasoning, we are going to investigate this method above the others. Previous implementation of ternary systems using the CMOS logic has provided proof that there is potential for increases in speed over binary systems. However, in previous systems, researchers have used an imbalanced ternary model using 0, 1, and 2[2] instead of a balanced ternary model using –1, 0, and 1.[3] Alternatively, the researchers have used a simple ripple adder instead of a more complicated adder structure. A ripple adder is one where each value must be determined in series, while a more complex adder uses parallel computation to speed up operations. In this paper we seek to answer the following question: If a ternary adder was built on the same formula as a binary adder using a

balanced ternary CMOS implementation, would that operate faster at comparable sizes to a binary adder of the same formula?

Modern-day computers are impressive, but the industry is always looking for improvements. A system that dramatically reduces calculations and can represent much larger numbers will always be worth exploration.

1.2. Result summary

Overall, the results of this research were within expectations. Each of the systems produced serial delays within the expectation of ternary performing better than binary. Additionally, each of the circuit schematic results took a correlating amount of time to the serial delays that were calculated for the expected results. These results, as shown later in this paper, are examples of the potential improvements that are possible with this base-3 system.

2. BACKGROUND

2.1. The problem

2.1.1. Binary Bias

The computers of the modern day operate on the same binary system that they have operated on since the advent of the modern computer. This has led to binary systems being far more developed than any potential alternatives. There is an inherent bias towards incumbent systems in development. The research and development required to transition to other devices is considerable, because of this and despite potential improvements, any other system is unlikely to completely replace the current commonplace binary device.

2.1.2. Why base-3?

It is natural to ask why one might use base-3. Several helpful measurements for determining the best radix for computations are the following: computational efficiency, circuit propagation delay, and circuit size. Computational efficiency can be defined as a measurement of rw where r is the radix and w is the width in digits while keeping r^w constant. The solution to this evaluation becomes clearer when we treat r and w as continuous rather than integers. The optimal value for r is e the base of natural logarithms.[4] As e is not a practical value to represent numbers due to its irrational nature, we look at the nearest neighbors to e, these being 2 and 3.

We can see, using an example from Hayes's article, $999{,}999_{10}$, has an r of 10 and a w of 6, giving us $rw_{10} = 60$. For base-2 we have 11110100001000111111, which is w of 20, r of 2 giving us $rw_2 = 40$. Balanced ternary splits the values on zero, so we use –1, 0, 1 instead of 0, 1, 2. For the purpose of this paper, –1 will be represented by T. In base-3 we have 1212210202000 in imbalanced ternary and 1T0T0T11T1T000 in balanced ternary. One can see that for imbalanced ternary we have w of 13 and r of 3 giving us $rw_3 = 39$. In this case for Balanced ternary however we have w of 14 and r of 3 giving us $rw_3 = 42$. This is because while balanced and imbalanced ternary represent the same number of values, balanced ternary ranges from $-\frac{3^n-1}{2}$ to $\frac{3^n-1}{2}$ and imbalanced ternary represents from 0 to $3^n - 1$. Due to this one might think that imbalanced ternary is the superior option, however the complexity added when subtracting numbers makes the imbalanced ternary option less efficient than the ternary option because of the needed inclusion of a signed trit reducing the usable trits.

Using another example, one can see that with $9{,}999{,}999_{10}$, $rw_{10} = 70$; $1001\ 1000\ 1001\ 0110\ 0111\ 1111_2$, $rw_2 = 48$; $200\ 211\ 001\ 102\ 100_{3imb}$ our $rw_{3imb} = 45$; and $1\ T01\ T11\ 001\ 11T\ 100_{3bal}$ our

rw_{3bal} = 48. Here, imbalanced ternary is still scoring a lower and therefore better score than the other options, but binary and balanced ternary are tied. The larger the number gets, using ternary becomes more efficient in comparison to using binary.

Consider that in a computing system you frequently need to store the two (or three)'s compliments so you can subtract. You also need to reserve a bit for the sign in both binary and imbalanced ternary. This causes an increase in width(w) by 1, but our balanced ternary doesn't change. For a given number x, $rw_2=2 \cdot (\log_2 x + 1)$, $rw_{3imb}=3 \cdot (\log_3 x + 1)$, and $rw_{3bal}=3 \cdot (\log_3 2x)$. For larger signed numbers balanced ternary will always be more efficient as $rw_{3imb} < rw_2$ for $x >= 1$ and $rw_{3bal} < rw_{3imb} < rw_2$ for $x >= 1$ or $x <= -1$.

2.1.3. Why Not Base-3?

If base-3 is a better base for computation, why aren't we using it now? Hayes posits in his article the following:

> Once binary technology became established, the tremendous investment in methods for fabricating binary chips would have overwhelmed any small theoretical advantage of other bases. Furthermore, it's only a hypothesis that such an advantage exists. Everything hinges on the assumption that rw is a proper measure of hardware complexity, or in other words that the incremental cost of increasing the radix is the same as the incremental cost of increasing the number of digits.[4]

The question of whether or not *rw* is a meaningful measurement of hardware complexity or in other words, that ternary provides an advantage over binary on the hardware level, is something this paper seeks to answer. Additionally, the challenge in swapping manufacturing from one system

to another is important to consider. Those challenges are different based on the method of ternary implementation one would consider, and for some, the benefits gained by the ternary system may not be large enough to be worth adopting the new system.

2.2. Potential solutions

There are multiple potential solutions that have been explored or are being explored when it comes to ternary computational devices. The first of which was developed in the USSR in the late 1950s; others are more recent with development of CMOS, CNTFET, and Optical systems. We will explore here the first implementation of ternary logic as well as three of the modern options with the most potential for development.

2.2.1. Setun

One of the first systems to be developed using base-3 logic was in the USSR between the years 1958 and 1965. These Setun devices, named for a nearby river, used balanced ternary logic. However, the Setun devices did not use the ternary logic to its fullest as they stored information in pairs of magnetic cores, each of which could've been used to instead represent two bits in a binary system.[4] This concept was a key motivation for this project.

2.2.2. CNTFET

Carbon nanotube field effect transistors (CNTFETs) are an emerging research field as the development of nanotechnology advances. CNTFETs are expected to function more efficiently at lower voltages and space than CMOS devices. However, the intricacy of the connections required is

still a challenge for the performance and energy efficiency to be improved. This option also has significant issues in manufacturing regarding the difficulty of gate creation and alignment, as well as difficulties with compatibility with other advances in technology.

2.2.3. Optical

Another system for a ternary computer is being developed by a team in Shanghai, China. This Optical Ternary Computer uses orthogonally polarized light to represent the two different active states of a balanced ternary system. The device uses liquid crystal and light filters to create its gates and store its values. While their development has been promising, it faces a major hurdle in larger development: costs. The Optical Ternary Computer requires precise alignments and specialized pieces to be created. The manufacturing process does not exist for these in the refined and widespread way that it does for other methods of representing ternary computers.

2.2.4. CMOS

Modern computers use CMOS circuits to perform their operations. Similar CMOS circuits can be used to create a ternary system. This can be done in one of two ways. The first implements an imbalanced ternary system, the second implements a balanced ternary system.

The first CMOS method, which makes an imbalanced ternary system, uses resistors to create a stable middle point between full voltage and ground.

The second CMOS method, which makes a balanced ternary system, uses resistors and Zener diodes to create a stable zero point between two oppositely charged power lines. This is achieved by splitting a power source to create a ground that is in the middle of the two sources

where one is a positive and the other is negative voltage, speaking relatively. These CMOS circuits use a pair of n- and p-channel MOSFET transistors connected to these opposite voltages (see Figure 1).

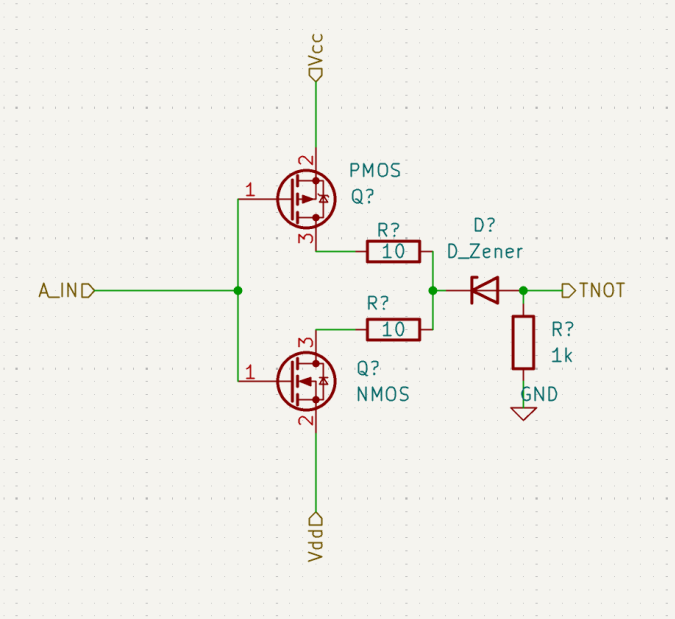


Figure 1: An image showcasing the CMOS circuit for a Negate gate.

These transistor pairs can be used to create the basic gates required for ternary logic. These basic gates are Negate, N-Any, N-And, N-Or, and N-Cons. The Negate gate inverts the input from 1 to –1 and –1 to 1 and leaves 0 at 0. The N-Any gate, also known as the Negated-Any gate, returns the input invert if either input is 1 or –1 but not opposites, and 0 if both inputs are 0 or if the inputs are opposites. The N-And gate, also known as the Negated And gate, returns the inverted result of the minimum, that is if either input is negative, it returns positive but only if both inputs are positive does it return negative. It returns 0 is both are 0 and if only one result is positive, but neither is negative. The N-Or gate, also known as Negated-Or gate, returns on the opposite operation of N-And where it returns negative if either input is positive, positive only if both inputs are negative, it returns 0 is both are 0 and if only one result is negative, but neither is positive. The N-Cons, also

known as the Negated-Consensus gate returns the inverted result if the two inputs are the same and 0 in all other situations.[3]

Ternary CMOS may not be the best option, though. As pointed out by Zahoor et al, the CNTFET option is preferred for its easily adjustable threshold voltages.[2] However, as CNTFETs have issues with scale and leakage currents, and their manufacturing process is more costly than that of a CMOS system, it is pertinent that we first explore the options granted by ternary CMOS technology.

2.3. Literature Review

Ternary computer systems have been developed for decades. From Setun[4] to the original multi-valued CMOS devices to carbon nanotube technology, people have been trying to develop ternary options for computers. Each step taken provides the grounds for future developments.[2]

Optical Ternary computers remain an option that should be researched further. The efficiency of these devices and the speed that they could theoretically operate at mean that they are an incredible resource that should be developed. However, at this current moment, there are too many barriers to their manufacturing for them to be considered on a large scale. There are other ternary options that should be used as steppingstones for the transition to ternary computation while the manufacturing process for optical ternary devices is researched and developed.

Ternary CMOS devices are one such option that have been explored multiple times. In more modern research, designs for an imbalanced Ternary full adder have been designed by Jo et al. [5][6] Their adder used a total of 68 transistors in 34 pairs of CMOS devices. A CMOS pair is a pair of N-channel and P-channel Metal-Oxide-Semiconductor Field-Effect Transistors (MOSFETs), so each pair is two transistors. Duret-Robert was able to create a more efficient design for a balanced ternary

adder that uses only 44 transistors for a full adder. According to Duret-Robert's research this full adder outpaces a binary full adder with a comparison of 7*n*-6 layers of serial propagation delay for the binary ripple adder, where n is the number of bits. This stores a smaller max number per n, the number of bits or trits, compared to 9*n*-7 layers for the ternary ripple adder, which stores a larger max number per n.

These numbers are better compared when the propagation delay is scaled by the largest number that can be displayed by *n* trits, where a trit refers to the ternary equivalent to a bit. The way to show this is to define *n* as the $\log_r(x)$ and compare the curves. The graph in Figure 2 shows this outpacing, alternatively the outpacing is evident as $9 \cdot \log_3(x) - 7$, our 9*n*-7 layers from before, is smaller than $7 \cdot \log_2(x) - 6$. A binary ripple adder using pass transistor logic (PTL) has a propagation delay of 4*n*-3 CMOS layers. While this method of logic is not typically applied in modern computers, it does show the kind of technological advances that binary systems have achieved. Comparing the graphs of the ternary ripple and the binary PTL shows the binary PTL on average performs better than the ternary.[3]

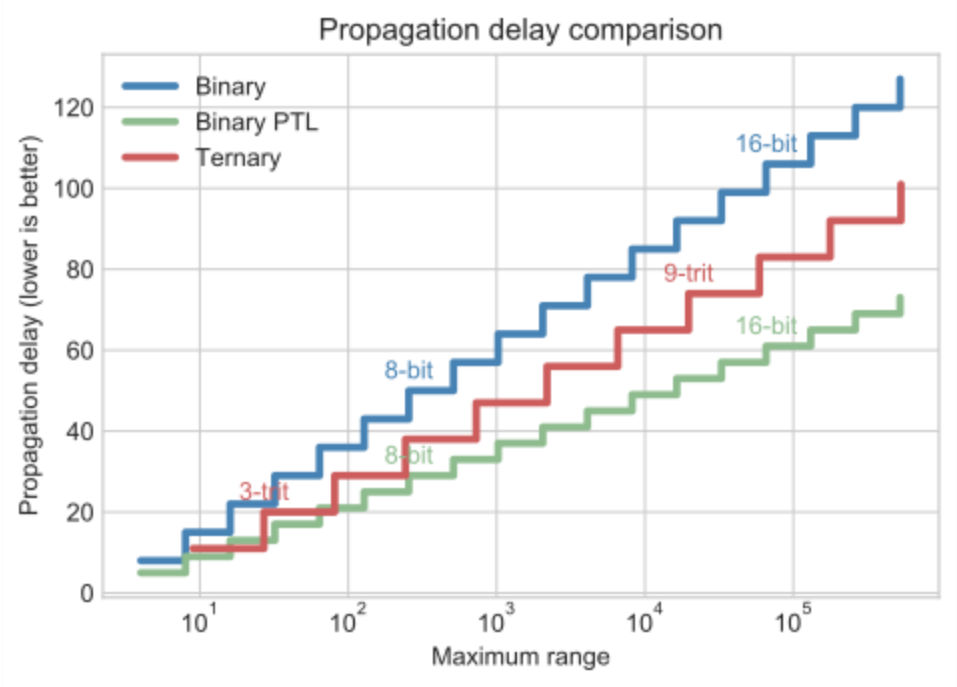


Figure 2: A graph from Duret-Robert's research showcasing propagation delay

With PTL and other advancements in binary computation, it is easy to assume that binary computation is superior. However, some advancements have been made in modern ternary computing as well, which makes the two more comparable. A series of more advanced adders than the basic ripple adder was created in ternary using the CNTFET style. These adders were compared to the binary equivalent and proved to be faster and more energy efficient. The only caveat remains that they are made with CNTFETs and are not easy to reproduce. Additionally, the study containing the details of this adder was only published in April of 2025, limiting the amount of time for it to have an impact. It is important to note that the advanced adders in the above-mentioned study were for an imbalanced ternary model, and not a balanced model.[7]

There have been other studies into the subject claiming that advanced adders in ternary are possible and effective. One such study describes a ternary carry lookahead adder for an imbalanced model. It, however, does not have a physical or simulated implementation and is purely theoretical.[8]

3. METHODOLOGY

There were two methodologies explored during the course of this study. The initial plan was more reminiscent of Setun, representing ternary systems with paired bits. This idea was initially conceived from the research conclusion by the team that made the Ternary Optical device, as they argued that manufacturing devices that were closer to the modern binary computer would be an important stepping-stone towards ternary computers. [1] The second plan was to use the CMOS based system to create a balanced ternary design that would be able to compare more adequately with modern binary systems. The adder that was chosen for this task was a Brent-Kung Adder (BKA). This was due to its reduced complexity as concerns in previous research were raised about the complexity of an advanced ternary adder.[3][8]

The initial plan consisted of a series of implementations where systems of binary gates were used to manipulate the pairs of bits that would represent the balanced trit. The 10_2, 00_2, and 01_2 states would be used but 11_2 would be an illegal state. This design came with an intrinsic understanding that the advantage was lost in this system. However, the intention was to use the system as a temporary stepping-stone while manufacturing for better systems, such as Optical or CNTFET ternary devices, was improved. The paired bit system was chosen because it could represent a balanced trit by treating the two bits. The preference for a balanced system led to this decision. This plan provided the groundwork for the second and primary implementation in this research.

Duret-Robert's research described a method of using CMOS transistor pairs, resistors, and a Zener diode to create balanced ternary gates. For these ternary gates they used a system with a positive and negative voltage with the 0 being the ground point. To achieve the negative and

positive voltage is relatively simple. Instead of having a single power source, you either wire two power sources in series or, if working with a single power source as is often the case when working with computers, either using a resistor divider[9] or a DC-DC converter.[10] Then, your middle point becomes ground, one side is negative, and the other is positive.

Using the CMOS method, which in other research had only been used for imbalanced ternary, seemed to be a significantly better option than the initial plan. My advisor and I agreed to swap focus and scale from the development of a ternary computational device using paired bits to developing plans for a better adder for the CMOS method of representing balanced ternary.

For this, it was determined that the best option for an improved adder would be the Brent-Kung Adder. The deciding factors for this were primarily the operation speed and the circuit complexity. While the Kogge-Stone Adder has a better operation speed, it has a higher complexity. While Brent-Kung Adders take $2 \cdot \log_2(n) - 2$ layers of operation logic, Kogge-Stone Adders take $\log_2(n)$.[11] That difference is significant, but Brent-Kung Adders have a smaller chip area size than Kogge-Stone Adders.[12][13] This was the main reason that Brent-Kung was chosen over Kogge-Stone for this project, especially because the concern of overcomplication in ternary devices.[8]

Towards this method two solutions were devised. The first solution was a simple simulation of gate operations in Python. The second solution was a more complex simulation using the KiCAD circuit simulator. For this we used the designs of the Brent-Kung adder from its initial design[14] and the designs that Duret-Robert used in their designs for the basic adders.[3]

4. PRESENTATION OF WORK

4.1. Simulations

The Python simulation made use of only a few functions and basic binary operations for the binary adders. Each function for the ternary gates increased a global variable known as 'pairsused' that tracked the number of CMOS pairs that were required for each gate so that the total number of pairs used for an adder was clear. A simple gate was any operation that required only one Zener diode and pull-down resistor. A complex gate was any operation that layered multiple simple or complex gates together. The simple ternary gates were Negate, N-Any, and N-Cons. Unimplemented were N-And and N-Or. The complex ternary gates were Cons, Any, Add, and FullAdd. Unimplemented was Mult. The functions are included in Appendix A.

Within this Python file, the simulations were run for similar sizes of storage (3^9 and 2^{16}) and determined the following information:

Table 1 Python Simulation Counts

Adder	Base	Width	Transistor Pairs	Serial layers of delay for MSB
Ripple	2	16	288	112
BKA	2	16	284	31
Ripple	3	9	194	52
BKA	3	9	247	16

It is evident from the Python simulation that the ternary Brent-Kung adder is better than any of the other options in terms of speed, where speed is equated to a lower number of serial layers. The ternary ripple adder is the best in terms of least used transistors.

While nine trits and sixteen bits are similar in size, they are not exact. Additionally, sixteen bits holds more than balanced nine trits, so it was important to determine the layers of delay that were required per bit or trit n to see how the systems would scale. To this end, the following four equations were derived from the simulation, including tests of smaller simulated scales.

Table 2 Serial Delay Formula

Adder	Ripple	Brent-Kung
Binary	$7n$	$4 \cdot Floor(2 \cdot \log_2(n)) - 1$
Ternary	$6n$-2	$2 \cdot Floor(2 \cdot \log_2(n)) + 4$

The circuit simulations were used as real-world examples of the operations performed in the Python simulations. While the circuit simulations are not perfect recreations of the real-world evaluations, they are better representations than the Python simulations.

Ten different circuits were simulated: A ripple and BKA for each of the following; 8-bit, 16-bit, 5-trit, 9-trit, and 10-trit. These simulations were used to confirm correlation between the above formulas and the practical evaluation speeds of the adders.

KiCAD was chosen for the simulators for its free and intuitive software. It also uses the standard nspice simulations for its circuit simulations making it practical and accurate software to be used for the development and simulations of the circuits. The simulations for the Negate, N-Any, N-Cons, and Add were created initially using only pure simulation parts, but were quickly switched to MOSFETs in the 2n7002k family. These gate designs were developed by Duret-Robert, though I used my own design for the Add gate that was based on their design. My Add gate gives back both the carry generate and propagate, or the add and the carry. The difference depended on if the Add gate was being used for the ripple or BKA.

The ripple adders used two Add gates and an Any gate for each trit after the first. Combined, this meant that each trit added 6 serial layers of delay except for the first which added only 4, as reflected in the earlier formulas. The Brent-Kung adders were more complicated with the initial propagate and generate being performed using the Add gate which occurred in parallel. The next steps were identical to the steps of a normal Brent-Kung adder, but instead of using OR and AND gates a series of N-Any, N-Cons, and Negate gates were used. These steps were combined following

the original design for a Brent-Kung adder with trial and error begin used to finalize the pattern to optimize the layers of serial propagation delay.

Initially, the system used the Consensus gates in replacement for the And gates and Any gates in replacement for the OR gates, but a newer solution was found and was able to use the negated forms of these gates to reduce the propagation delay.

The gates themselves were implemented with certain specifications. Each gate had three power input flags, V_{cc},V_{dd}, and GND. For the sake of the simulation, the power was assumed to have already been split into positive and negative from a standard 12 volt power supply. For each gate, these three labels were wired to +6v, -6v, and 0v. Each gate also had either one input A, or two inputs A, B. Except for the FullAdd gate which took an additional C input for the carry. For each gate, two stabilizing resistors of 10 Ohms were used, while a pull-down resistor of 1 Kiloohm was used after the Zener diode to ensure a stable 0v third state.

In comparison, Duret-Robert, whose work I based my gates on, used 1k stabilizing resistors and 100k pull-down. I found in my simulation that those numbers were unnecessary, though the ratio between the two was important. There's evidence that a version without the stabilizing resistors may be possible, but it is not within the scope of this project.

For the binary adders, the simplest implementations of each of Xor, Not, And, and Or were used. They used the same resistors and transistors as the Ternary gates but implemented power on a smaller scale of 0v to +6v. The structure of the Ripple adder matched that of Duret-Robert's research, and the structure of the Brent-Kung adder matched that of the original Brent-Kung adder scaled to 16-bit architecture.

There were multiple experiments run using these specifications to find the comparisons between the different adders and see how they compared to one another and the Python program. It

is important to note that all of the gates for all of the adders, both binary and ternary, used the same resistors and same 2n7002k family MOSFETs. Trials were run on dozens of inputs to determine which inputs would be simulated as the worst case for each adder design.

The worst case found for the Ternary Brent-Kung Adder was A as all 1, B as alternating 1 and 0. This resulted in the same as the negative flip of A as all –1 and B as alternating –1 and 0. The worst case found for the ternary Ripple adder was A as all 1 and B as the first trit being 1 and all others being 0. The worst case found for the binary adders were both A as all 1 and B as all 0 except for the first bit which was 1.

The full designs are available in Appendix B.

4.2. Summary of results

The results were clear: ternary is faster than binary. The expected results were reflected in the trial data, and the ternary system operated with a lower propagation delay than the binary system. The delay in the KiCAD nspice simulations matched very closely with the Python expectations for the delay. The ten different adders all tested on their worst case found gave the following microsecond delays.

Table 3 Circuit Simulation Microsecond Delays

Adder	Size	Microsecond To Complete
Binary Ripple	8-Bit	36
Binary Ripple	16-Bit	70
Ternary Ripple	5-Trit	29
Ternary Ripple	9-Trit	46
Ternary Ripple	10-Trit	51
Binary Brent-Kung	8-Bit	21
Binary Brent-Kung	16-Bit	30
Ternary Brent-Kung	5-Trit	13

Ternary Brent-Kung	9-Trit	17
Ternary Brent-Kung	10-Trit	18

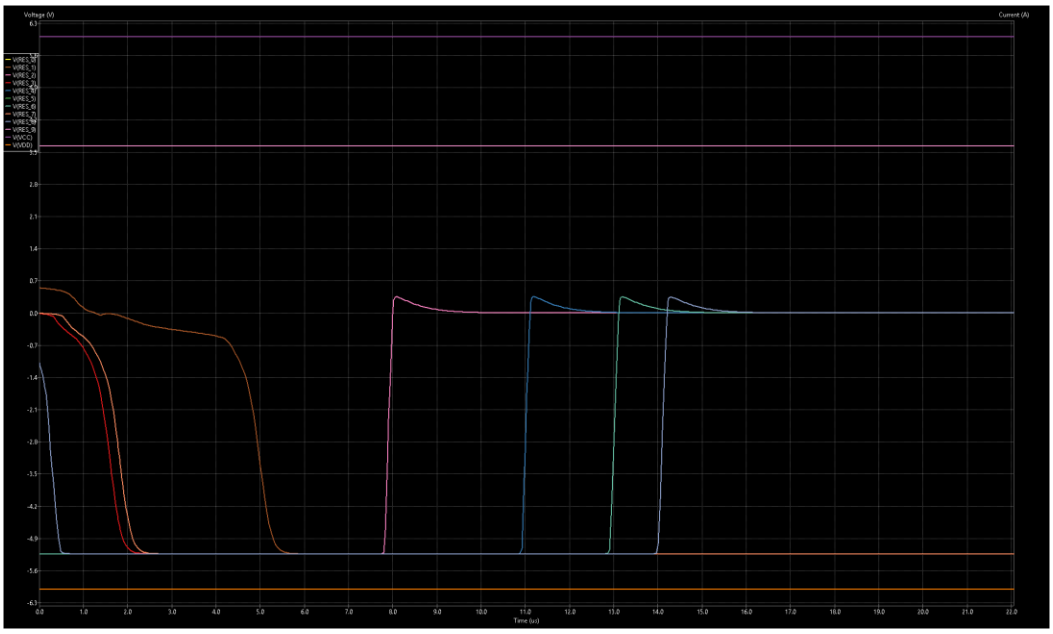


Figure 3: Circuit simulation of a 9-trit BKA, showing resulting voltages for each trit.

These results correlate closely with the formulas derived earlier. Those formulas show that the progression of operable space for a ternary device expands exponentially faster than that of binary device with no delay due to the complex operations. Instead, ternary devices are faster and able to store and perform operations for larger numbers.

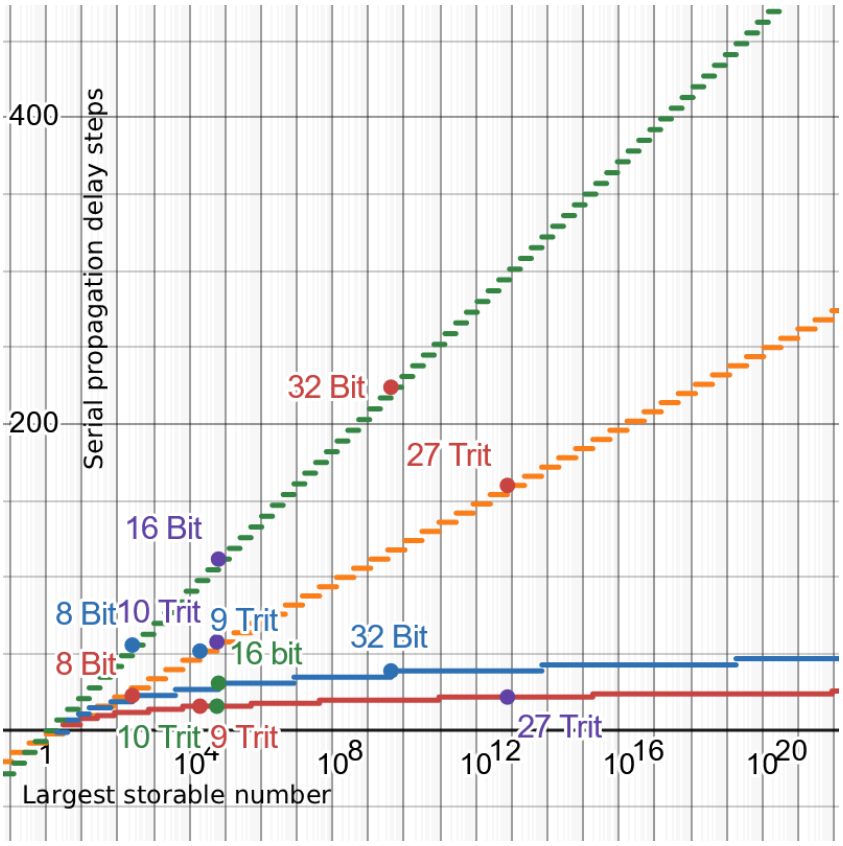


Figure 4: Formula comparisons between the different adders showing the propagation delay in serial steps

5. CONCLUSIONS

5.1. Recommendations

From the results, it is clear that the ternary system is faster and more efficient than the binary system. Ideally, the continuation of this research would be exploration into exactly what resistor strengths, transistors, and voltages would be preferred for the optimal operation speeds. Additionally, exploration into full ternary computational devices implemented with the CMOS ternary design would be of value.

5.2. Future Work

Future work includes developing more optimized adders specifically designed for a ternary system. Additionally, work would be done to develop a ternary system that can perform the operations of a more advanced computer. Currently, the most advanced ternary device is the Shanghai team's ternary optical computer, but using this CMOS design there is an opportunity to develop a more advanced ternary device that can compare to a modern binary computer. Work would need to be done to implement the BKA structure on the scale of more trits and see how the device compares at 64-trit to a modern 64-bit Computer given a balanced 64-Trit computer has a lower theoretical propagation delay and can store numbers from $-\frac{3^{64}-1}{2}$ to $\frac{3^{64}-1}{2}$.

5.3. Conclusion

The Brent-Kung Adder implemented in balanced ternary shows that the base-3 has the potential to function equivalent to or faster than their base-2 counterparts. Further research into these possibilities is necessary to fully explore the opportunities presented by the ternary technology. Development of lower-trit fully computer systems should be investigated and compared to their binary equivalents first. Then higher trit values should be utilized to determine how a full system compares to the modern binary system. Each of these options and more are valuable avenues of research that should be investigated.

REFERENCES

1. H. Wang, S. Ouyang, Y. Shen and X. Chen, "Ternary Optical Computer: An Overview and Recent Developments," 2021 12th International Symposium on Parallel Architectures, Algorithms and Programming (PAAP), Xi'an, China, 2021, pp. 82-87, doi: 10.1109/PAAP54281.2021.9720446.
2. F. Zahoor et al., "Design implementations of ternary logic systems: A critical review," Results in Engineering, vol. 23, p. 102761, Sep. 2024, doi: https://doi.org/10.1016/j.rineng.2024.102761.
3. "Ternary ALU," Github.io, 2019. https://louis-dr.github.io/ternalu3.html (accessed Oct. 13, 2025).
4. B. Hayes, "Computing Science: Third Base," American Scientist, vol. 89, no. 6, pp. 490–494, 2001, doi: https://doi.org/10.2307/27857554.
5. J. Ko, K. Park, S. Yong, T. Jeong, T. H. Kim and T. Song, "An Optimal Design Methodology of Ternary Logic in Iso-device Ternary CMOS," 2021 IEEE 51st International Symposium on Multiple-Valued Logic (ISMVL), Nur-sultan, Kazakhstan, 2021, pp. 189-194, doi: 10.1109/ISMVL51352.2021.00040.
6. J. Ko, J. Kim, T. Jeong, J. Jeong and T. Song, "Exploration of Ternary Logic Using T-CMOS for Circuit-Level Design," in IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 70, no. 9, pp. 3612-3624, Sept. 2023, doi: 10.1109/TCSI.2023.3287274.
7. S. V. Yamani, H. K. R. Vamsi Kudulla, B. V. R. S. Ganesh, D. Sushma, C. Manasa, and S. H. Prasad, "Design an energy efficient ternary parallel prefix carry/sum propagate adders using 32-nm CNTFET," Memories - Materials, Devices, Circuits and Systems, vol. 10, p. 100129, Mar. 2025, doi: https://doi.org/10.1016/j.memori.2025.100129.

8. "Douglas W. Jones on Ternary Arithmetic," Uiowa.edu, 2015. https://homepage.cs.uiowa.edu/~dwjones/ternary/arith.shtml (accessed Oct. 13, 2025).
9. DigiKey Electronics, "How to Obtain Positive and Negative Voltage Rails from a Single Output Power Supply," DigiKey TechForum - An Electronic Component and Engineering Solution Forum, Jan. 31, 2024. https://forum.digikey.com/t/how-to-obtain-positive-and-negative-voltage-rails-from-a-single-output-power-supply/38446 (accessed Oct. 13, 2025).
10. nateolson, "Get Positive and Negative Voltage from Single Power Supply," All About Circuits, Nov. 29, 2022. https://forum.allaboutcircuits.com/threads/get-positive-and-negative-voltage-from-single-power-supply.190632/ (accessed Oct. 13, 2025).
11. "How to add numbers (part 2)," Lag.net, 2025. https://robey.lag.net/2012/11/14/how-to-add-numbers-2.html (accessed Oct. 13, 2025).
12. I. Koren, (2002). Computer Arithmetic Algorithms. AK Peters Series. Massachusetts:A K Peters.
13. V Sidharthan and P. Mani, "Comparative Analysis of Adders Parallel-Prefix Adder for Their Area, Delay and Power Consumption," ResearchGate, Apr. 2021, doi: https://doi.org/10.13140/RG.2.2.23867.95529.
14. R P Brent, H T Kung, "A REGULAR LAYOUT FOR PARALLEL ADDERS," June 1979. http://www.dtic.mil/get-tr-doc/pdf?AD=ADA074455, Archived at: https://web.archive.org/web/20170924184528/http://www.dtic.mil/get-tr-doc/pdf?AD=ADA074455 (accessed Oct. 13, 2025)

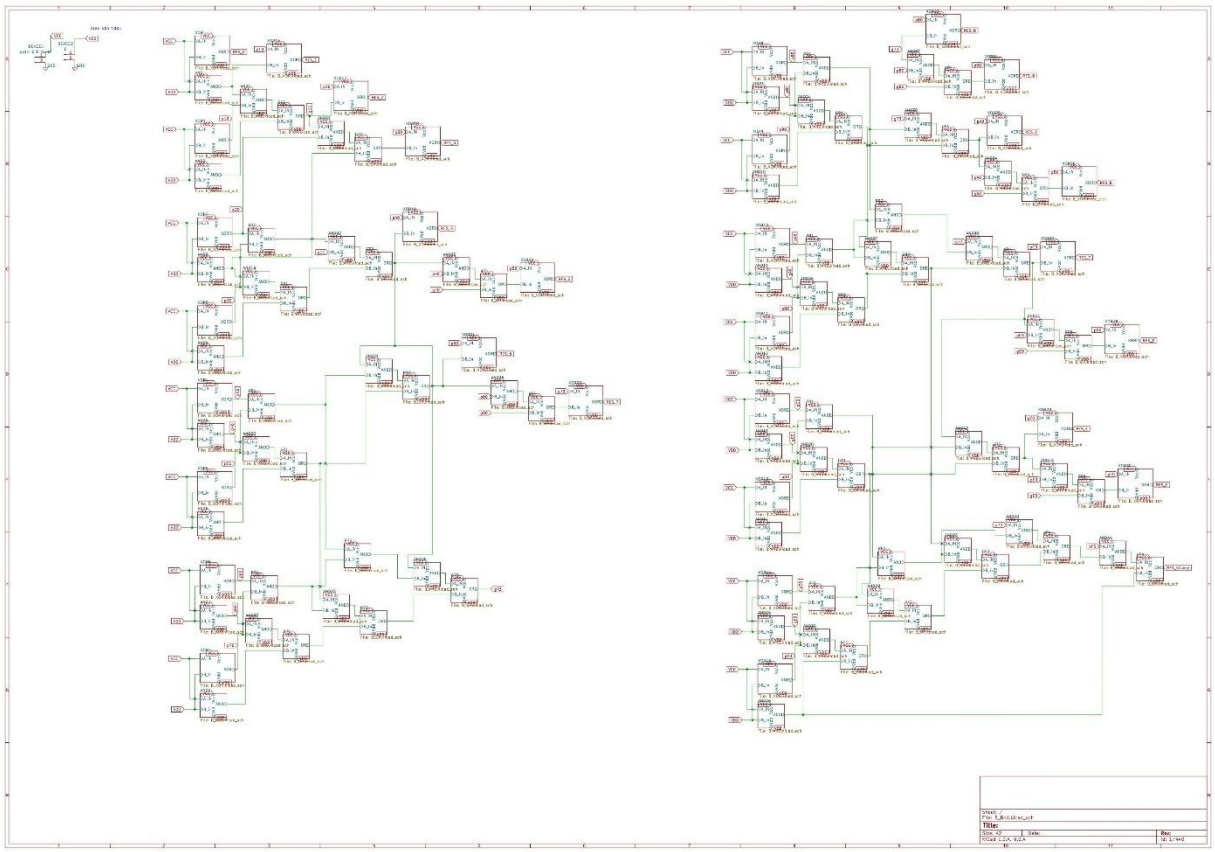
APPENDICES

Appendix A: Python Functions

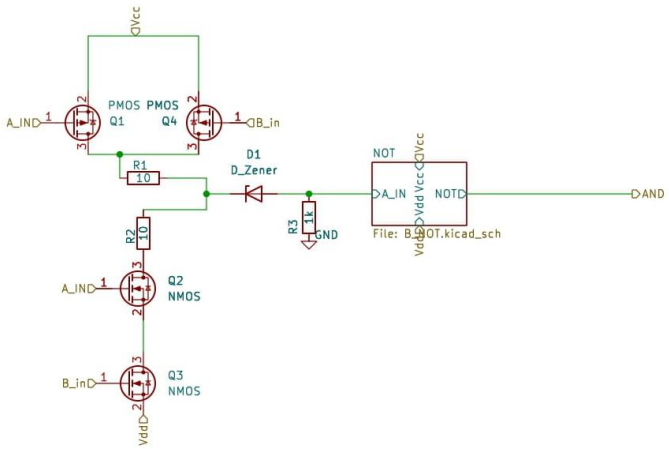
```
 def NEG(a):
    global pairsused
    pairsused += 1
    return a * -1

def NANY(a,b):
    global pairsused
    pairsused += 2
    if a == b * -1:
        return 0
    elif a != 0:
        return a * -1
    elif b != 0:
        return b * -1

def NCONS(a,*b):
    global pairsused
    pairsused += 1 + len(b)
    if a == b[0] and len(set(b)) == 1:
        return a * -1
    else:
        return 0

def CONS(a,*b):
    # 2 layers
    global pairsused
    pairsused += 2 + len(b)
    if a == b[0] and len(set(b)) == 1:
        return a
    else:
        return 0

def ANY(a,b):
    # 2 layers
    return NEG(NANY(a,b))

def ADD(a,b):
    # 4 layers
    innerWire1 = NCONS(a,b)
    wireOut1 = NEG(innerWire1)
    wireOut2 = NANY(NANY(innerWire1,ANY(a,b)), wireOut1)
    return wireOut1, wireOut2

def ADDFULL(a,b,cin):
```

```
# 10 layers, 4 for each add, 2 for any
carry1, sum1 = ADD(a,b)
carry2, sum2 = ADD(sum1,cin)
carryout = ANY(carry1,carry2)
return carryout, sum2
```

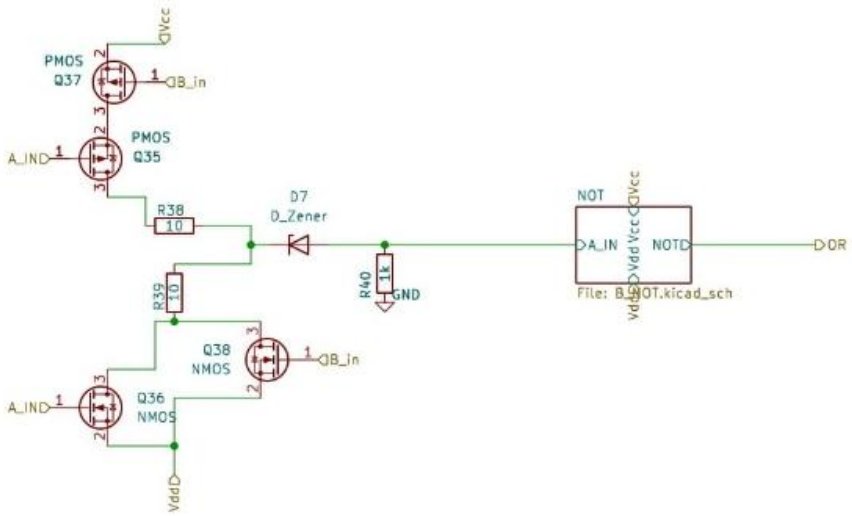
Appendix B: Circuit Designs



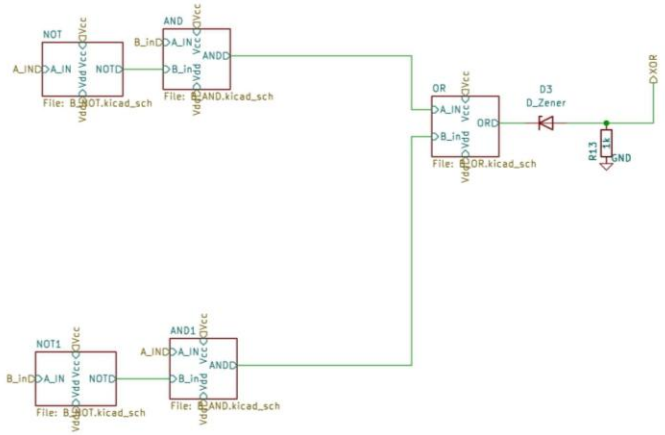
Binary BKA 16-Bit Circuit



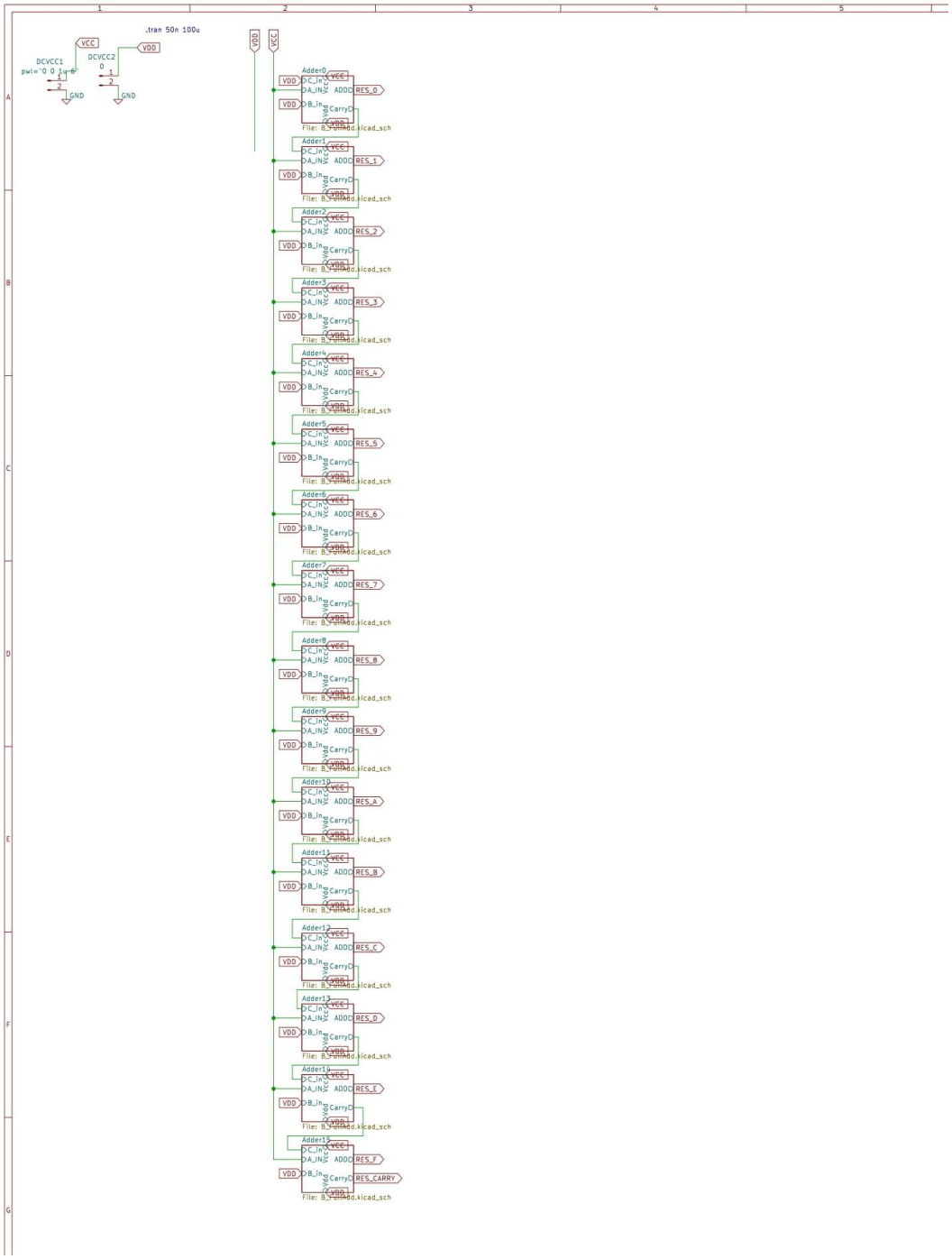
Binary AND Circuit



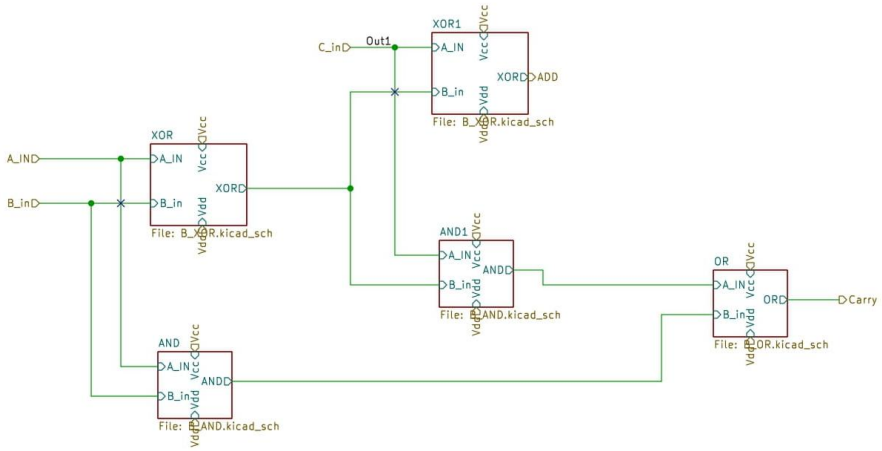
Binary OR Circuit



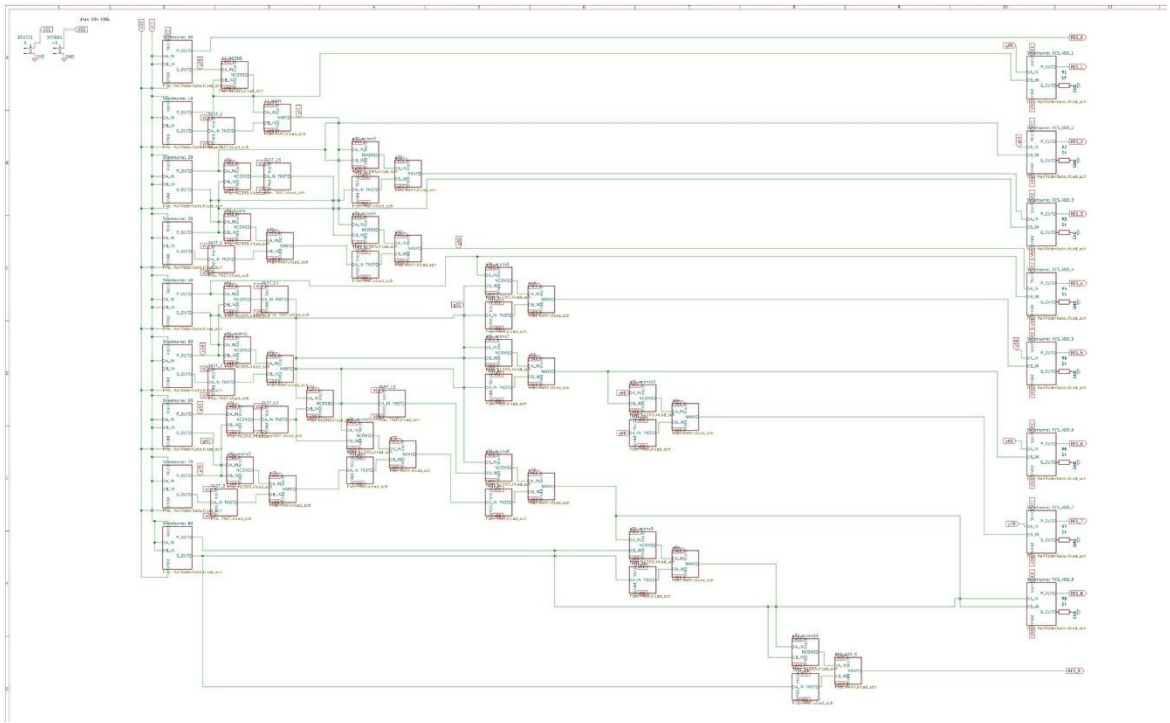
Binary XOR Circuit



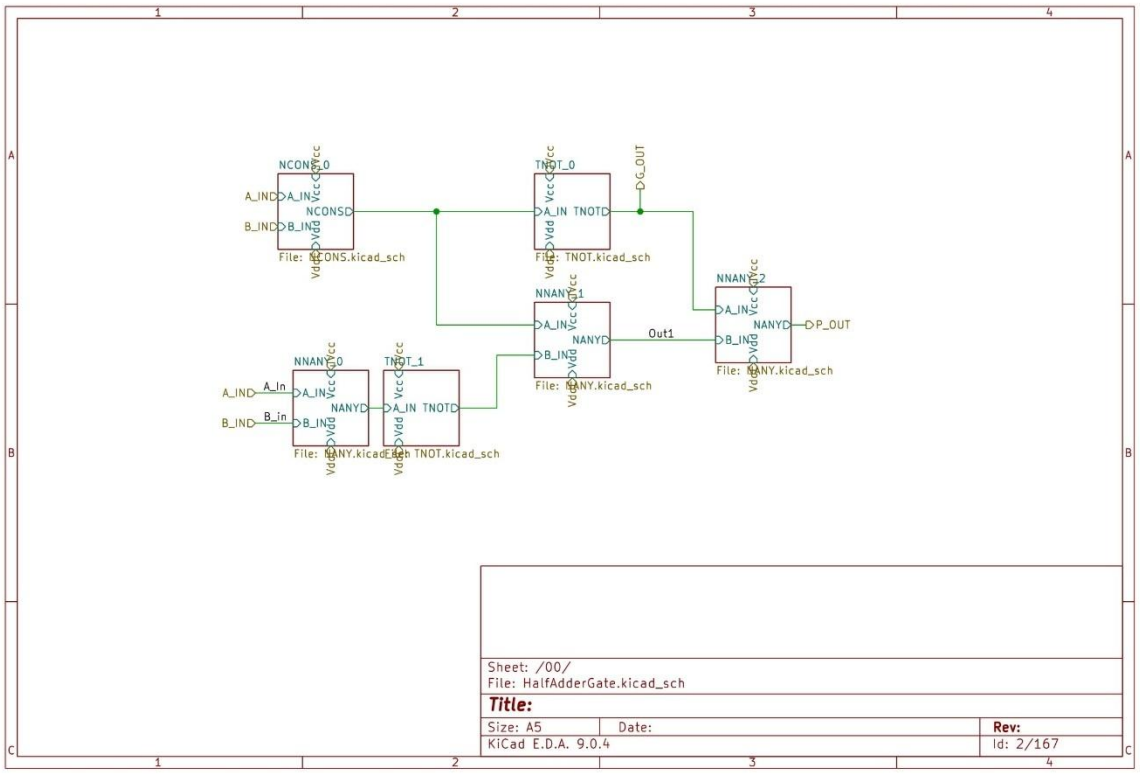
Binary Ripple Adder 16-bit Circuit



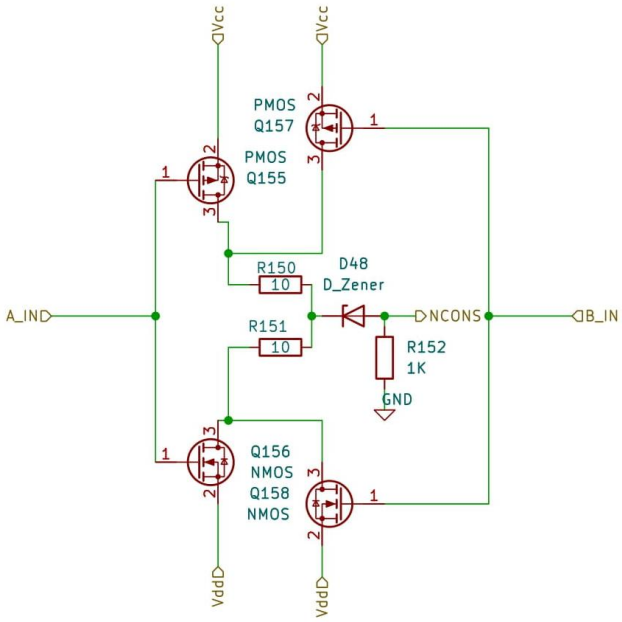
Binary ADD Circuit



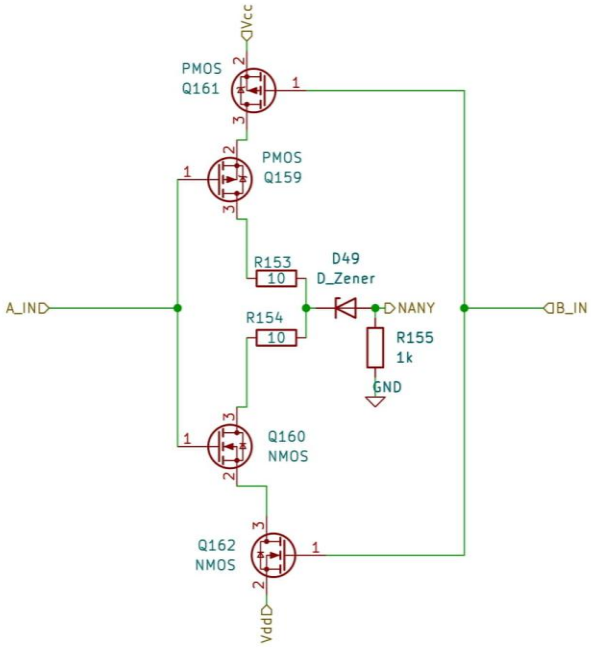
Ternary BKA 9-trit Circuit



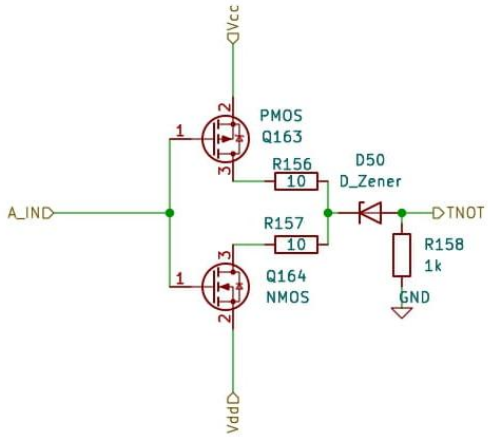
Ternary Half Adder 1-trit Circuit



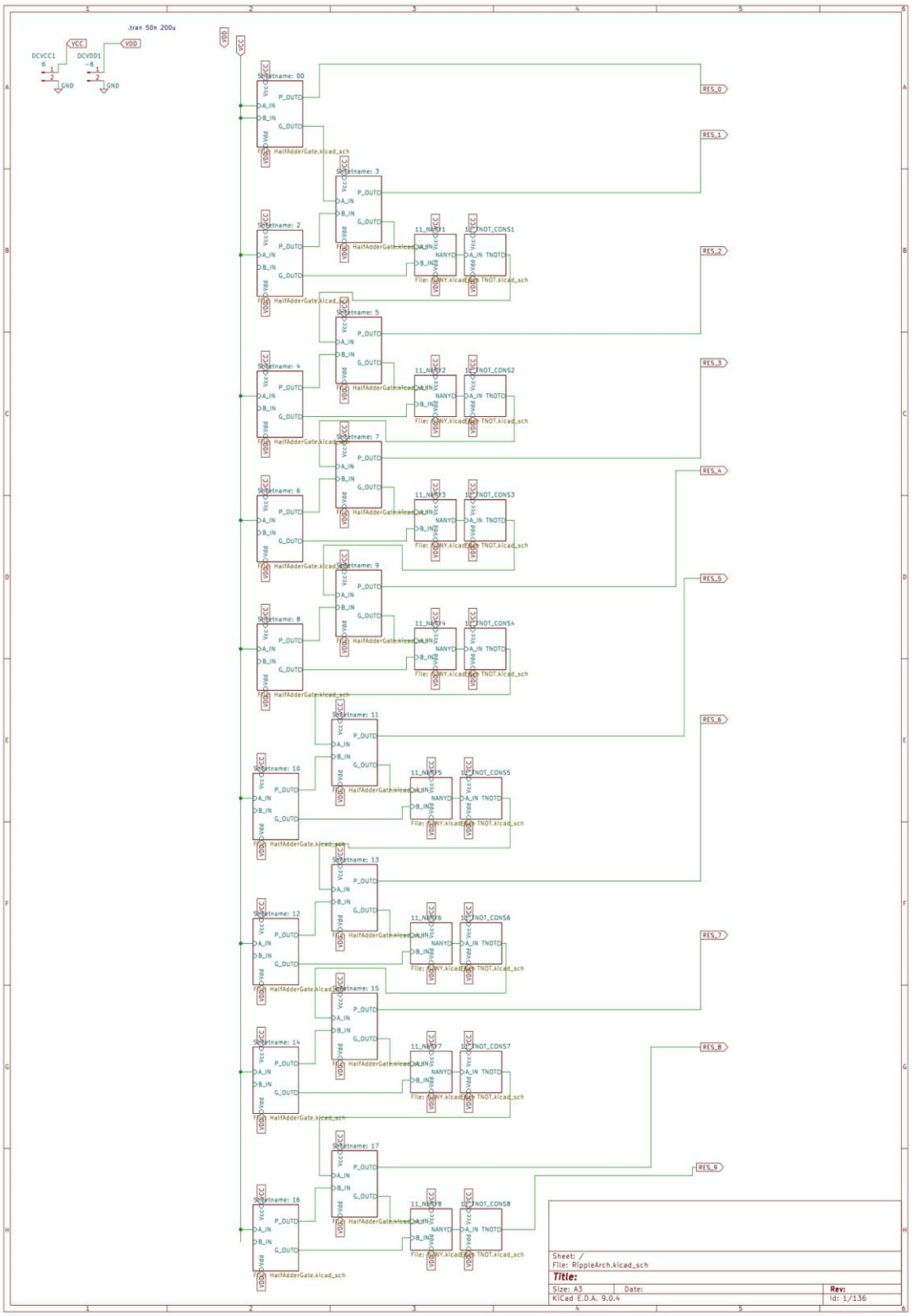
Ternary NCONS Circuit



Ternary NANY Circuit



Ternary NEGATE Circuit



Ternary Ripple Adder 9-trit Circuit

SIGNATURE PAGE

EXPLORING BRENT-KUNG ADDERS IN BALANCED TERNARY CMOS LOGIC

A thesis submitted to the D. Abbott Turner College of Business and Technology in partial fulfillment of the requirements for the degree of

MASTER OF SCIENCE

TSYS SCHOOL OF COMPUTER SCIENCE

by

Astrea J.C. Rojas 2025

Dr. Hyrum D. Carroll, Chair — Date 11/18/2025

Dr. Lixin Wang, Member — Date 11/18/2025

Dr. Rodrigo Obando, Member — Date 11/18/2025